Inteligență Artificială

Regresia Logistică

Slides: Mihai Trăscău

- Regresia Logistică

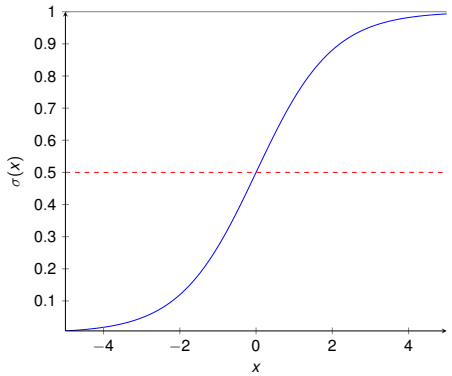
- Clasificare multi-clasă

- Regresia logistică este o metodă de învățare supervizată
- Este utilizată în probleme de clasificare
- În general folosită pentru probleme de clasificare binară
- Se poate extinde și la clasificare multi-clasă

- Am putea utiliza regresia liniară?
  $h_\theta(x) \geq 0.5 \Rightarrow y = 1$
  $h_\theta(x) \leq 0.5 \Rightarrow y = 0$
- Nu este o idee bună! (De ce?)

- Dorim ca $0 \leq h_\theta(x) \leq 1$
- Utilizăm funcția sigmoid care asigură valori de ieșire în intervalul $[0, 1]$

$$\sigma(x) = \frac{1}{1 + e^{-x}} = \frac{e^x}{1 + e^x}$$



- Astfel, vom calcula ipoteza h ca fiind o combinație liniară a atributelor de la intrare peste care aplicăm funcția sigmoid

$$h_\theta(x) = \sigma(\theta^T x)$$

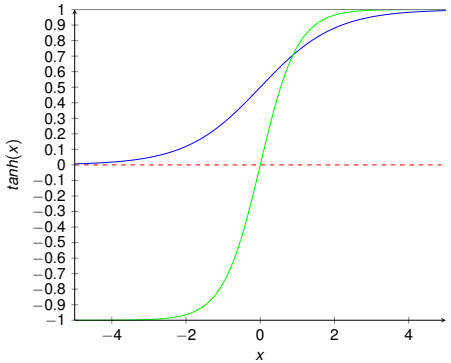
adică

$$h_\theta(x) = \frac{1}{1 + e^{-\theta^T x}}$$

- $h_\theta(x)$ estimează probabilitatea ca $y = 1$ pentru un x dat, adică $h_\theta(x) = P(y = 1|x, \theta)$

- Este posibil să utilizăm și alte funcții similare, precum tangenta hiperbolică (*tanh)* care asigură valori de ieșire în intervalul $[-1, 1]$

$$tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$



- Avem $0 \leq h_\theta(x) \leq 1$ și putem să considerăm (pentru simplificare) cazul cu 2 atribute, x_1 și x_2, pentru care calculăm probabilitatea clasei y, adică $p = P(y = 1|x, \theta)$
- Funcția logit este inversa funcției sigmoid:

$$logit(x) = \sigma^{-1}(x) = ln(\frac{x}{1-x})$$

Atunci,

$$\frac{p}{1-p} = e^{\theta_0+\theta_1 x_1+\theta_2 x_2}$$

Deci,

$$\boxed{p = \frac{1}{1+e^{-(\theta_0+\theta_1 x_1+\theta_2 x_2)}}}$$

- Ce funcție de cost să utilizăm?
- Pentru regresia liniară am utilizat eroarea pătratică medie (MSE):

$$J(\theta_0, \theta_1, ..., \theta_m) = \frac{1}{2n} \sum_{i=1}^{n} \left(h(x^{(i)}) - y^{(i)} \right)^2$$

- Pentru regresia liniară avem $h_\theta(x) = \theta^T x$, iar pentru regresia logistică $h_\theta(x) = \sigma(\theta^T x)$
- Problemă: Funcția de cost (MSE) nu ar mai fi convexă (nu mai are un minim global unic)
- Soluție: Utilizăm altă funcție de cost!

- Funcția de cost: **Plauzibilitatea maximă (Maximum Likelihood)**
- Reținem din Teorema lui Bayes că:

$$\textbf{APosteriori} = \frac{\textbf{Plauzibilitate} \cdot \textbf{APriori}}{\textbf{Probe}}$$

Considerând θ – parametrii modelului și $\mathbf{X} = \{x^{(1)}, x^{(2)}, \ldots, x^{(n)}\}$ – setul de învățare (dataset) punem problema ca:

$$\max_{\theta} p(\theta|\mathbf{X}) = \frac{p(\mathbf{X}|\theta) \cdot p(\theta)}{p(\mathbf{X})} = \max_{\theta} p(\mathbf{X}|\theta) \cdot p(\theta)$$

Dacă presupunem că $p(\theta)$ este o **distribuție uniformă**, atunci **maximizarea aposteriori a parametrilor modelului** se reduce la **maximizarea plauzibilității**:

$$\max_{\theta} p(\mathbf{X}|\theta)$$

- Cum exprimăm $p(\mathbf{X}|\theta)$?
- Considerăm setul de învățare ca fiind **i.i.d.** (eng. **independently and identically distributed**)

$$p(\mathbf{X}|\theta) = \prod_{i=1}^{n} p(x^{(i)}|\theta) = \prod_{i=1}^{n} h_\theta(x^{(i)})^{y^{(i)}} \left(1 - h_\theta(x^{(i)})\right)^{1-y^{(i)}}$$

unde am utilizat faptul că $y^{(i)} \in \{0, 1\}$ și că $h_\theta(x^{(i)}) = P(y^{(i)} = 1|x^{(i)}, \theta)$

- În practică, produsul multor valori de probabilitate poate să ajungă o valoare instabilă numeric, iar bibliotecile de optimizare lucrează frecvent cu probleme de **minimizare**
- Astfel, **logaritmăm** și **inversăm semnul** expresiei pentru a ajunge la forma de **minimizare** a **Negative Log Likelihood (NLL)**

$$NLL = -log(p(\mathbf{X}|\theta)) = -\sum_{i=1}^{n} \left[y^{(i)} log(h_\theta(x^{(i)})) + (1 - y^{(i)}) log(1 - h_\theta(x^{(i)})) \right]$$

Sumarizând:

- Ce funcție de cost să utilizăm?

$$Cost(h_\theta(x), y) = \begin{cases} -log(h_\theta(x)), & \text{if } y = 1 \\ -log(1 - h_\theta(x)), & \text{if } y = 0 \end{cases}$$

cu scrierea compactă,

$$Cost(h_\theta(x), y) = -ylog(h_\theta(x)) - (1 - y)log(1 - h_\theta(x))$$

Deci,

$$J(\theta) = -\frac{1}{2n} \sum_{i=1}^{n} \left[y^{(i)} log(h_\theta(x^{(i)})) + (1 - y^{(i)}) log(1 - h_\theta(x^{(i)})) \right]$$

unde alegem pentru funcția de cost o medie a NLL-ului

- Pentru a face predicții trebuie să găsim parametrii θ care minimizează funcția de eroare:

$$\underset{\theta}{argmin}\, J(\theta)$$

- Derivata parțială a funcției de eroare este:

$$\frac{\partial J(\theta)}{\partial \theta} = \frac{1}{n} \sum_{i=1}^{n} \left(h_\theta(x^{(i)}) - y^{(i)} \right) x^{(i)}$$

- Putem utiliza algoritmul de scădere a gradientului (similar ca la regresia liniară)

Gradient-Descent($\Theta_{init}, \alpha, \epsilon$) – caută minimul funcției $J(\theta)$

Intrări: Θ_{init} – valorile inițiale
α – rata de învățare
ϵ – prag pentru criteriul de convergență

1. $\Theta^{(0)} \leftarrow \Theta_{init}$
2. $t \leftarrow 0$
3. **repetă**
4. $\quad t \leftarrow t + 1$
5. $\quad$ **pentru fiecare** $\theta_j \in \Theta$
6. $\quad\quad \theta_j^{(t)} = \theta_j^{(t-1)} - \alpha \frac{1}{n} \sum_{i=1}^{n} \left(h(x^{(i)}) - y^{(i)} \right) x^{(i)}$
7. **până când** $|f(\Theta^{(t)}) - f(\Theta^{(t-1)})| < \epsilon$
8. **întoarce** $\Theta^{(t)}$

- Problema de optimizare pentru găsirea parametrilor θ se poate face și cu alți algoritmi:
  - Scăderea conjugată a gradientului
  - Broyden-Fletcher-Goldfarb-Shannp (BFGS) - metodă quasi-Newtoniană
  - Limited Memory Broyden-Fletcher-Goldfarb-Shannp (L-BFGS)

- Regresie liniară
  - Distribuția predicțiilor și erorilor este Gaussiană
  - Valorile prezise sunt continue
- Regresie logistică
  - Distribuția este Bernoulli/Binomială
  - Valorile prezise sunt probabilități pentru apartenență (sau non-apartenență) la o clasă

- Tipuri de distribuții:
  - Bernoulli - 1 eveniment cu ieșire $1/0$
  - Categorială - 1 eveniment cu k ieșiri
  - Binomială - n evenimente cu ieșire $1/0$
  - Multinomială - n evenimente cu k ieșiri

Clasificare multi-clasă

- Cum extindem regresia logistică la clasificare multi-clasă?
- Utilizăm conceptul *one-versus-all* pentru k clase
- Pentru fiecare clasă $i \in \{1, \ldots, k\}$ avem ipoteza $h_\theta^i(x) = P(y = i|x, \theta)$
- La inferență determinăm fiecare valoare $h_\theta^i(x)$ și alegem clasa i pentru care:

$$\underset{i}{argmax}\ h_\theta^i(x)$$

Clasificare multi-clasă : Tratare overfitting

- Cum minimizăm riscul de overfitting?
- Reducem numărului de atribute
  - Manual, prin selectarea și eliminarea lor efectivă
  - Prin aplicarea unor algoritmi de selecție a atributelor
  - Soluția reduce overfittingul dar poate duce la pierderea de informație utilă

Clasificare multi-clasă : Tratare overfitting

- Regularizare
  - Impunem constrângeri asupra valorilor parametrilor θ
  - Însumăm norma pătratică a vectorului de parametri θ ca termen suplimentar la funcția de eroare
  - Se forțează astfel găsirea unui set de parametri cu valori cât mai mici $\Rightarrow$ în procesul de optimizare, modelul va seta anumiți θ_j la valori mici
  - Atributele x_j corespunzătoare acestor θ_j devin astfel „ignorate” în inferență (se înmulțesc cu θ_j corespondent care este aproape 0)
- Funcția de eroare devine:

$$J(\theta) = -\frac{1}{2n}\sum_{i=1}^{n}\left[y^{(i)}log(h_\theta(x^{(i)})) + (1-y^{(i)})log(1-h_\theta(x^{(i)}))\right] + \lambda||\theta||^2$$

- unde λ este rata de regularizare, cu $\lambda \in [0, 1]$

Mulțumesc!



Feedbackul este binevenit!